

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

Fase 1: Procesamiento y Modelado Inicial - Equipo 19

Domingo 25 de octubre del 2026

José Antonio Vázquez Martínez	A01797208
Jesús Ángel Rincón Ruiz	A01793960
Erick Francisco Siller Ojeda	A01382929
Gustavo Alejandro Vázquez Rivas	A01611559

Operaciones de aprendizaje automático | TC5061

Introducción

En esta primera fase del proyecto de la materia Operación con Modelos de Machine Learning (TC5061), trabajamos en la construcción inicial del pipeline de MLOps para nuestro equipo. La idea central no es solamente entrenar un modelo rápido para ver qué métrica nos da, sino estructurar todo el proyecto de manera profesional desde el día uno, cuidando que los datos se puedan reproducir, que el código sea fácil de entender y que cualquier experimento quede bien registrado.

Para este trabajo seleccionamos el dataset abierto de MiBici, que es el sistema de bicicletas públicas de Guadalajara. Revisando cómo opera la red, notamos que el problema principal está en el desbalance de bicicletas en las estaciones durante las horas pico de entrada y salida laboral. Por eso decidimos enfocar la solución en predecir la demanda por hora en cada estación, de modo que el equipo operativo pueda mover las unidades antes de que las estaciones se queden vacías o saturadas. Para lograrlo, organizamos el repositorio con Cookiecutter Data Science, programamos módulos limpios en Python usando orientación a objetos, conectamos MLflow para versionar los datos y definimos claramente las tareas de cada integrante según los roles de MLOps.

1. Análisis de Requerimientos y Propuesta de Valor

MiBici publica mensualmente sus datos abiertos de viajes; tan solo en mayo de 2024 se registraron 410,806 viajes en 360 estaciones activas de la zona metropolitana de Guadalajara (el catálogo vigente incluye 484 estaciones). Al revisar el comportamiento de los viajes, vimos que la gente se mueve en horarios muy marcados (con picos a las 8:00 y a las 18:00 en días laborales). Esto ocasiona que en estaciones céntricas o cercanas a universidades falten bicicletas cuando todos quieren salir, o que no haya puertos libres para devolverlas. Actualmente el rebalanceo con camionetas se hace de forma reactiva, lo que genera costos altos y quejas constantes de los usuarios.

Para plantear el proyecto de forma ordenada utilizamos la herramienta Machine Learning Canvas, resumiendo los aspectos clave:

Bloque del Canvas	Definición para MiBici Guadalajara
Propuesta de Valor	Predecir la demanda de bicicletas por estación cada hora con 1 a 24 horas de anticipación. Esto permite mover las bicicletas con tiempo, bajando eventos de desabasto y mejorando el servicio.
Fuentes de Datos	Dataset abierto de MiBici con el registro mensual de viajes (fechas, duraciones, origen, destino y datos de usuario) junto con el catálogo oficial de estaciones y sus coordenadas.

Ingesta y Almacenamiento	Pipeline automatizado por lotes que guarda los datos originales en la carpeta data/raw sin tocarlos, procesa una versión limpia en data/interim y registra todo en MLflow con hashes de seguridad.
Variables (Features)	Variables de tiempo (hora, día de la semana, indicador de fin de semana), transformaciones cíclicas con seno y coseno, ubicación de la estación (latitud, longitud, capacidad) y rezagos de demanda previa.
Tarea de Machine Learning	Regresión supervisada para predecir la variable "demand_count", que representa el número total de extracciones de bicicletas en una estación durante una hora específica.
Decisiones Operativas	Si la demanda calculada supera el 80% de las bicicletas disponibles, se manda una camioneta a surtir la estación. Si la estación está llena y la demanda es baja, se retiran unidades para permitir recepciones.
Modo de Inferencia	Inferencia por lotes (Batch) que se ejecuta cada 60 minutos para planear las siguientes horas. La predicción para toda la red de estaciones tarda menos de 5 segundos.
Evaluación Offline	Partición cronológica por fecha (70% entrenamiento, 15% validación y 15% prueba) para no mezclar datos del futuro con el pasado. Evaluaremos con MAE, RMSE y coeficiente R2.
Monitoreo en Producción	Revisión constante de la disponibilidad de bicicletas en las estaciones y monitoreo de data drift para detectar si los patrones de viaje cambian por clima, obras viales o días festivos.
Restricciones y Riesgos	Protección estricta de la privacidad de los usuarios (no se usan nombres ni datos personales directos), control de fallas si no hay internet y reproducibilidad total del código.

2. Manipulación y Preparación de Datos

Los datos se obtienen directamente del portal de datos abiertos de MiBici: la clase MiBiciDataDownloader (src/data/downloader.py) descarga los viajes de mayo de 2024 (datos_abiertos_2024_05.csv, 410,806 registros) y el catálogo de estaciones vigente (nomenclatura_2026_09.csv, 484 estaciones), normaliza el encabezado de la columna de año de nacimiento, que viene con caracteres corruptos en el archivo original, y guarda ambos en data/raw. Para limpiar los datos crudos desarrollamos la clase MiBiciDataCleaner dentro del archivo src/data/cleaner.py. El objetivo fue establecer reglas sencillas y lógicas para quitar registros con errores sin perder información valiosa:

• Quitar duplicados: Si un viaje viene con el mismo identificador "Viaje_Id" más de una vez, dejamos solo uno.

• Duraciones negativas o nulas: 921 viajes tienen una fecha de término igual o anterior a la de inicio, lo que indica un error de registro, así que los descartamos.

• Viajes menores a 1 minuto: Cuando un usuario saca una bicicleta y la regresa de inmediato casi siempre es porque la llanta venía ponchada o la cadena rota, por lo que no cuenta como un viaje real.

• Viajes mayores a 2 horas (120 minutos): El servicio está pensado para trayectos cortos de menos de 45 minutos. El percentil 99 de la duración es de 31 minutos, por lo que los 96 viajes que pasan de 2 horas son casos extremos (bicicletas no ancladas correctamente, descuidos o multas) que distorsionan el modelo.

• Estaciones inexistentes: Comprobamos que el origen y destino existan en el catálogo oficial de MiBici.

• Datos demográficos: En los registros que no tenían año de nacimiento colocamos la mediana calculada del grupo, y los valores vacíos de género los dejamos identificados como desconocidos. Además, descartamos 84 registros con edades fuera del rango de 14 a 90 años (por ejemplo, un año de nacimiento capturado como 200).

Criterio de Limpieza	**Filas**	**Porcentaje**
Registros brutos iniciales (Raw)	410,806	100.00%
Viajes descartados con duración negativa o cero	921	0.22%
Viajes descartados menores a 60 segundos	14,033	3.42%
Viajes descartados mayores a 120 minutos (outliers)	96	0.02%
Registros descartados por estación inválida	0	0.00%
Registros descartados por edad inválida (fuera de 14 a 90 años)	84	0.02%
Registros finales limpios y validados (Interim)	**395,672**	**96.32%**

Con estos filtros conservamos el 96.32% de los datos y descartamos el 3.68% de registros inconsistentes; la mayor parte (14,033) corresponde a viajes menores a un minuto. No se encontraron Viaje_Id duplicados ni estaciones fuera del catálogo.

3. Exploración y Preprocesamiento de Datos (EDA)

Antes de pasar a generar variables para el modelo, revisamos las distribuciones y el comportamiento de los viajes mediante gráficas que guardamos en la carpeta reports/figures:

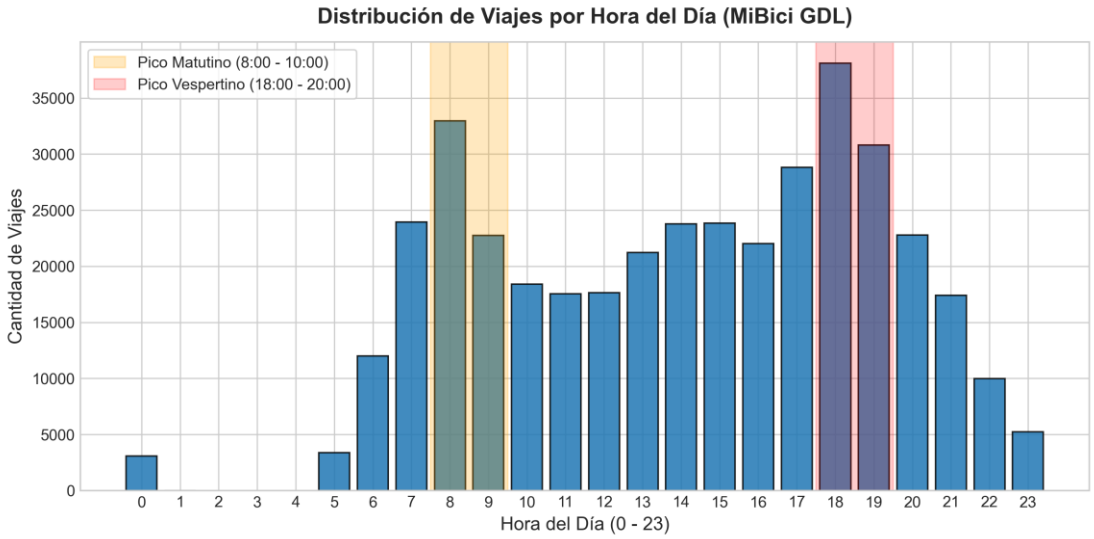


Figura 1. Distribución de viajes por hora del día en MiBici Guadalajara.
Nota: Se aprecian dos picos de demanda que coinciden con la entrada y salida laboral: 8:00 (32,962 viajes) y 18:00 (38,094 viajes). Entre la 1:00 y las 4:59 prácticamente no hay viajes porque el servicio no opera en ese horario.

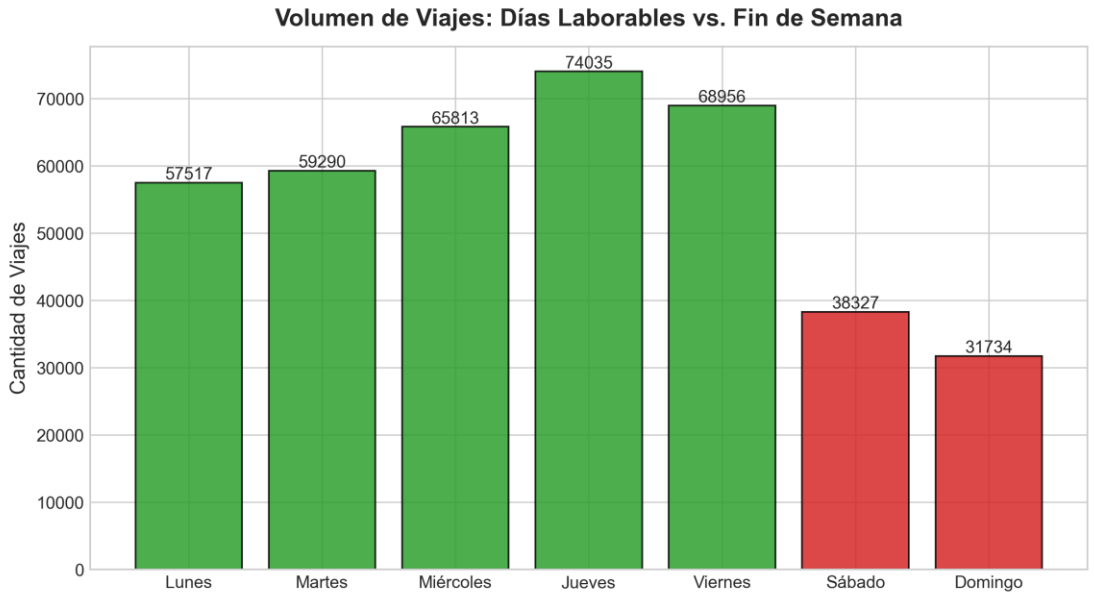


Figura 2. Volumen de viajes comparando días laborales contra fin de semana.
Nota: Los viajes se concentran de lunes a viernes (promedio de 65,122 por día de la semana) y caen alrededor de 46% en sábado y domingo (promedio de 35,031). Mayo de 2024 tuvo cinco miércoles, jueves y viernes, lo que eleva esos días. Esto confirma que la gente usa el sistema principalmente como transporte de trabajo y estudio.

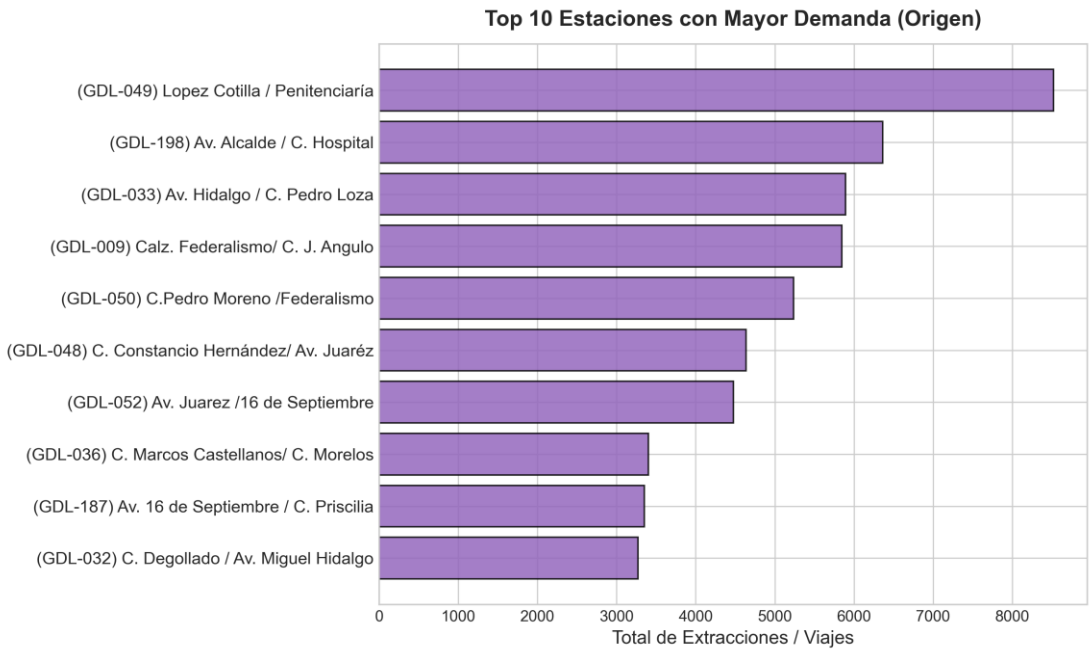


Figura 3. Las 10 estaciones de MiBici con mayor número de extracciones.
Nota: La demanda se concentra en el centro de Guadalajara: GDL-049 (López Cotilla / Penitenciaría) registra 8,523 extracciones, seguida de GDL-198 (Av. Alcalde / C. Hospital) con 6,365 y GDL-033 (Av. Hidalgo / C. Pedro Loza) con 5,895. Son las estaciones de atención prioritaria.

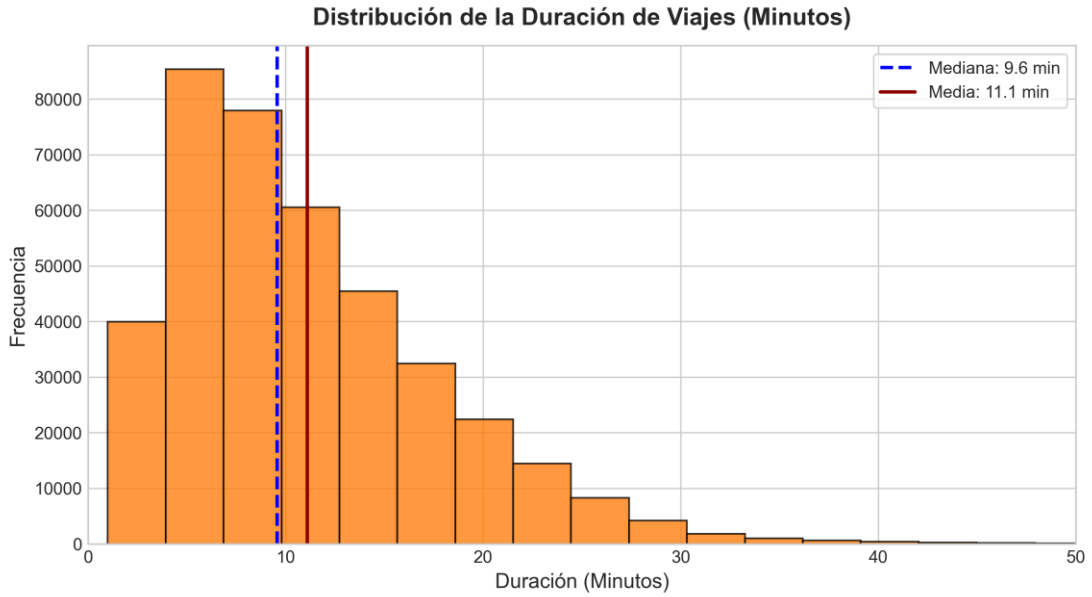


Figura 4. Distribución del tiempo de duración de los viajes en minutos.
Nota: La mediana de viaje es de 9.6 minutos y la media de 11.1 minutos, lo que confirma que el servicio se utiliza sobre todo para conectar trayectos cortos de última milla.

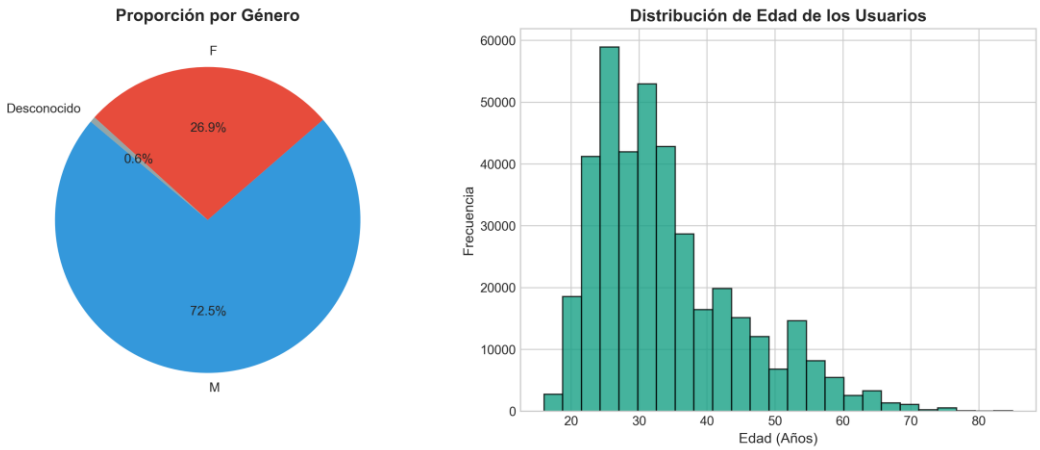


Figura 5. Perfil demográfico de los usuarios por género y edad.
Nota: El 72.5% de los viajes los realizan hombres, el 26.9% mujeres y el 0.6% no tiene género registrado. La mitad de los viajes corresponde a usuarios de entre 26 y 40 años (mediana de 31).

Ingeniería de Características (Feature Engineering)

Para preparar la matriz final creamos una cuadrícula completa que cruza las 360 estaciones activas en mayo de 2024 con las 744 horas del mes (267,840 filas). Si en una hora una estación no tuvo viajes, le asignamos demanda 0 para que el modelo aprenda a predecir también las horas tranquilas. El 50.9% de las combinaciones estación-hora tiene demanda cero y la demanda promedio es de 1.48 viajes por hora. Las variables que incluimos en el archivo data/processed/mibici_hourly_demand.csv son las siguientes:

1. Transformaciones de tiempo: Hora del día, día de la semana y una bandera para saber si es sábado o domingo.

2. Variables cíclicas: Como la hora 23 y la hora 0 están pegadas en el reloj, aplicamos seno y coseno (sin_hour, cos_hour, sin_dayofweek, cos_dayofweek) para que el modelo entienda esa continuidad.

3. Ubicación y tamaño de estación: Agregamos la latitud, la longitud y la capacidad total de anclajes de cada estación.

4. Rezagos de demanda (Lags): Incluimos la demanda de la hora anterior ("demand_lag_1h") y la demanda de la misma hora del día anterior ("demand_lag_24h"), ya que lo que pasó ayer a la misma hora ayuda mucho a predecir lo que pasará hoy.

Preprocesamiento y Modelos Baseline

Sobre la matriz de características aplicamos el preprocesamiento con la clase MiBiciPreprocessor (src/features/preprocessor.py), construida con Pipeline y ColumnTransformer de scikit-learn:

1. Partición cronológica: las primeras 520 horas del mes (1 al 22 de mayo, 187,200 filas) forman el conjunto de entrenamiento, las siguientes 112 horas el de validación (40,320 filas) y las

últimas 112 horas el de prueba (27 al 31 de mayo, 40,320 filas). Todos los transformadores se ajustan solo con entrenamiento para evitar fuga de información.

2. Normalización: StandardScaler sobre las variables cíclicas, las coordenadas, la capacidad y los rezagos de demanda.

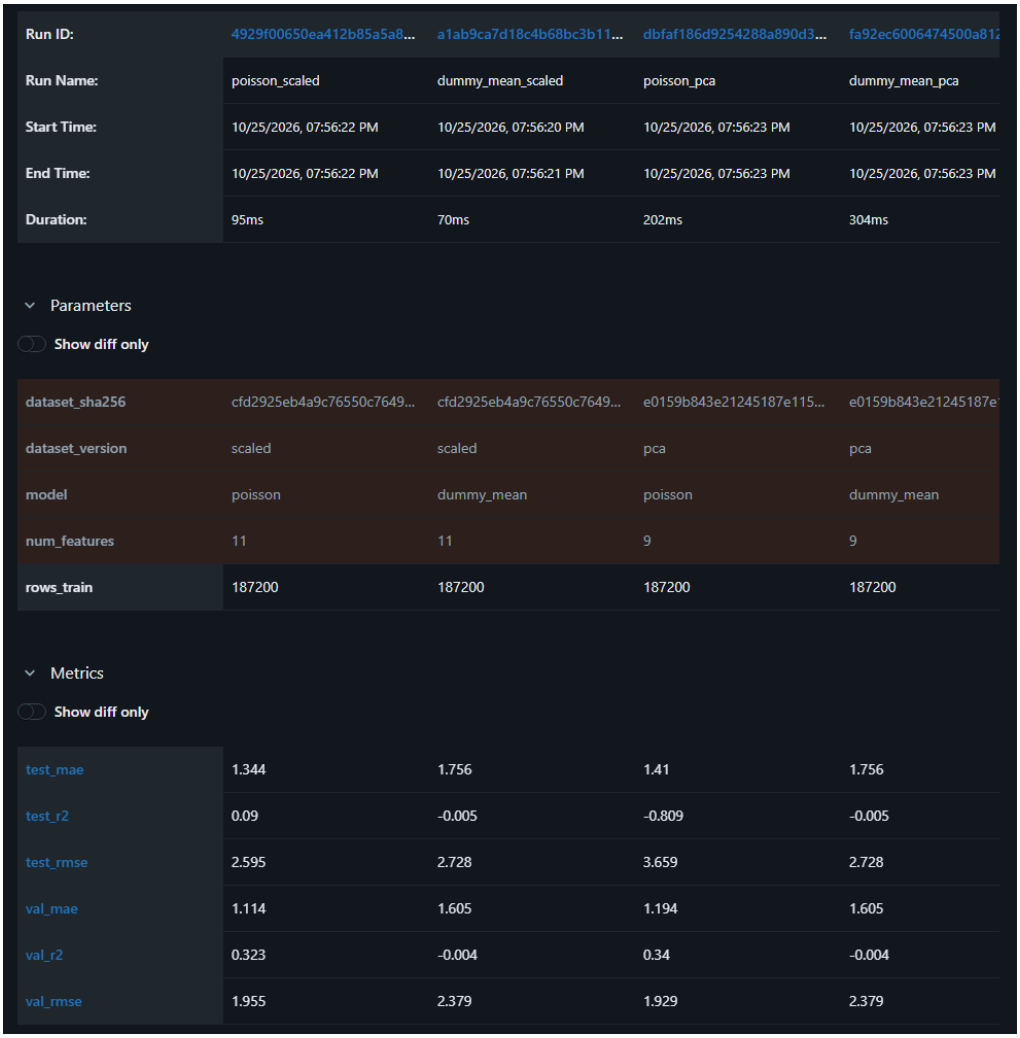
3. Codificación de variables categóricas: la estación (Origen_Id) tiene 360 categorías, por lo que one-hot generaría 360 columnas dispersas. Usamos TargetEncoder, que sustituye cada estación por su demanda media en entrenamiento con validación cruzada interna para no sobreajustar.

4. Reducción de dimensionalidad: PCA conservando el 95% de la varianza, que reduce las 11 variables a 9 componentes (95.75% de varianza explicada).

Cada versión (escalada y PCA) se guarda en data/processed y se registra en MLflow. Para experimentar con ellas entrenamos dos baselines con la clase BaselineTrainer (src/models/train_baseline.py): un DummyRegressor que predice la media y una regresión de Poisson, adecuada para conteos. Los resultados quedan en el experimento MiBici_Baselines_Fase1:

Modelo	Versión	MAE val.	MAE prueba	RMSE prueba	R2 prueba
Dummy (media)	scaled / pca	1.605	1.756	2.728	-0.005
Poisson	**scaled**	**1.114**	**1.344**	**2.595**	**0.090**
Poisson	pca	1.194	1.410	3.659	-0.809

La regresión de Poisson con la versión escalada reduce el MAE de prueba en 23% frente a la media (1.344 contra 1.756). La versión con PCA logra un MAE similar en validación, pero su RMSE de prueba es mucho mayor, lo que indica predicciones extremas en algunas horas. La diferencia entre validación y prueba se explica porque el conjunto de prueba solo contiene días laborales (demanda media de 1.67), mientras que validación incluye un fin de semana (1.32). Para la Fase 2 tomaremos como referencia la versión escalada y el MAE de 1.344 como línea base a superar.



Run ID:	4929f00650ea412b85a5a8...	a1ab9ca7d18c4b68bc3b11...	dbfaf186d9254288a890d3...	fa92ec6006474500a812
Run Name:	poisson_scaled	dummy_mean_scaled	poisson_pca	dummy_mean_pca
Start Time:	10/25/2026, 07:56:22 PM	10/25/2026, 07:56:20 PM	10/25/2026, 07:56:23 PM	10/25/2026, 07:56:23 PM
End Time:	10/25/2026, 07:56:22 PM	10/25/2026, 07:56:21 PM	10/25/2026, 07:56:23 PM	10/25/2026, 07:56:23 PM
Duration:	95ms	70ms	202ms	304ms

Parameters

Show diff only

dataset_sha256	cfd2925eb4a9c76550c7649...	cfd2925eb4a9c76550c7649...	e0159b843e21245187e115...	e0159b843e21245187e
dataset_version	scaled	scaled	pca	pca
model	poisson	dummy_mean	poisson	dummy_mean
num_features	11	11	9	9
rows_train	187200	187200	187200	187200

Metrics

Show diff only

test_mae	1.344	1.756	1.41	1.756
test_r2	0.09	-0.005	-0.809	-0.005
test_rmse	2.595	2.728	3.659	2.728
val_mae	1.114	1.605	1.194	1.605
val_r2	0.323	-0.004	0.34	-0.004
val_rmse	1.955	2.379	1.929	2.379

Figura 6. Comparación de las corridas baseline en MLflow UI.
Nota: Parámetros (hash SHA-256 del dataset, versión y modelo) y métricas de validación y prueba de las cuatro corridas del experimento MiBici_Baselines_Fase1.

4. Versionado de Datos con MLflow

Para asegurar que todo el trabajo sea reproducible, conectamos MLflow utilizando una base de datos local SQLite (mlruns.db). A través de nuestra clase MLflowDataVersioner (ubicada en src/utils/mlflow_logger.py), registramos cinco versiones de los datos: raw, interim, processed y las dos versiones preprocesadas (scaled y pca). Cada versión se vincula a su corrida como dataset de MLflow (mlflow.log_input). A cada archivo le calculamos su hash SHA-256 para tener la certeza de que nunca se modificó por error:

Nombre del Run	Etapa	Hash SHA-256 Digest	Filas	Rol
01_Raw_Data_Ingestion	raw	34cf4af18603bd20...	410,806	Data Engineer

02_Interim_Data_Cleaning	interim	b0828207af9665fa...	395,672	Data Engineer
03_Processed_Feature_Engineering	processed	c0a959070d98ecb5...	267,840	Data Scientist / MLE
04_Processed_Scaled_Encoded	processed_scaled	cfd2925eb4a9c765...	267,840	Data Scientist / MLE
05_Processed_PCA	processed_pca	e0159b843e212451...	267,840	Data Scientist / MLE

A continuación se muestran capturas reales de la interfaz web de MLflow UI (versión 3.17), donde se puede comprobar el registro de los experimentos, parámetros, métricas y los archivos guardados como artefactos:

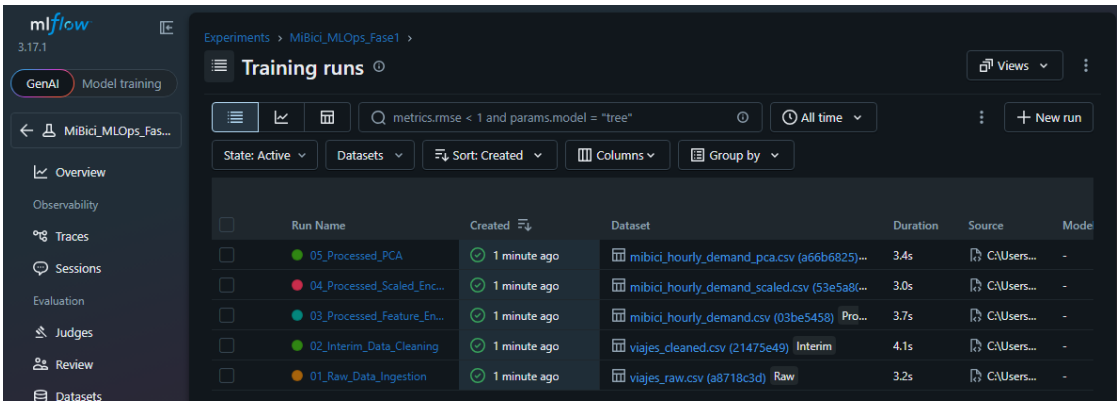


Figura 7. Vista general de ejecuciones (Runs) del experimento MiBici_MLOps_Fase1 en MLflow UI.
Nota: Vista Training runs con las cinco corridas registradas en SQLite (raw, interim, processed, processed_scaled y processed_pca), cada una con su dataset vinculado y su etapa.

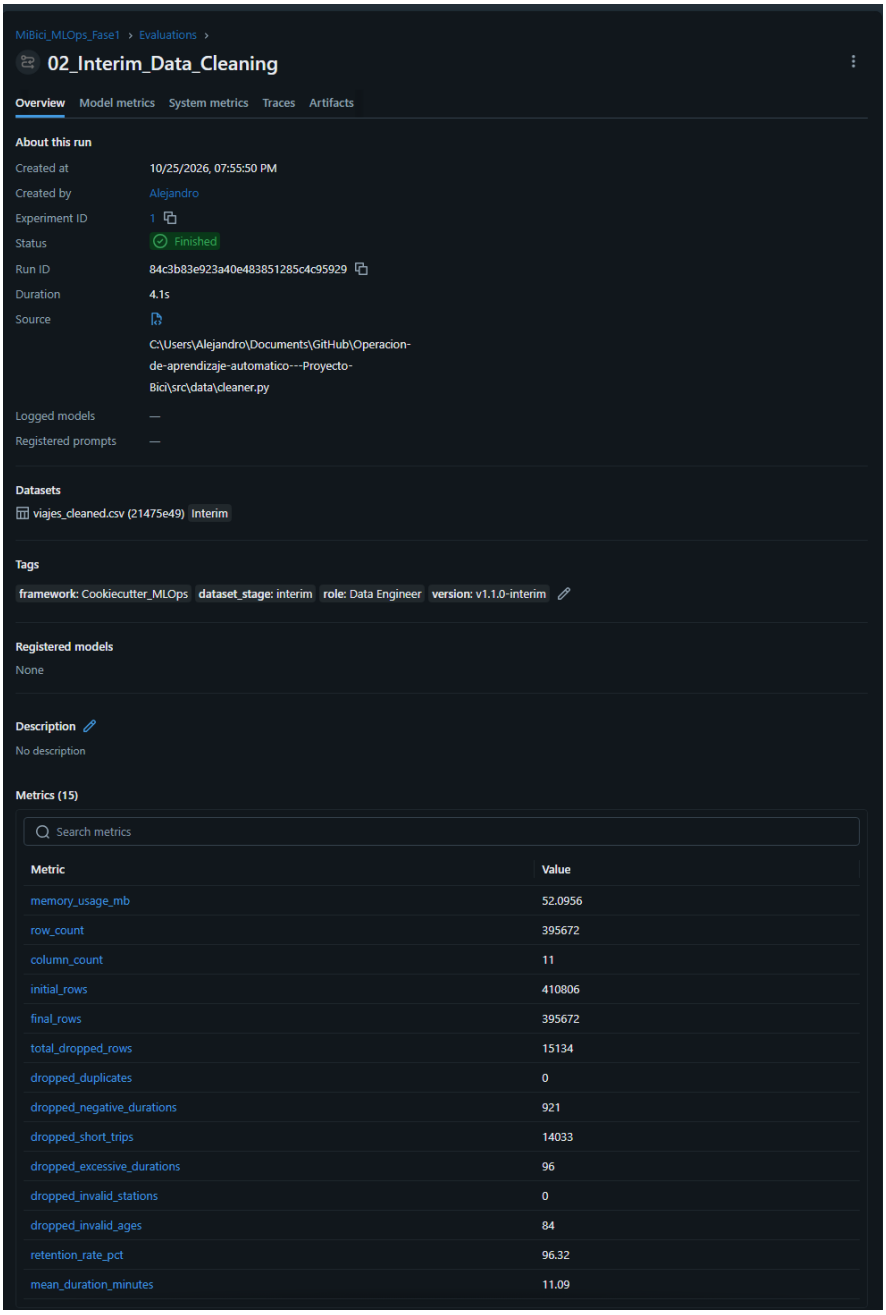


Figura 8. Detalle de la corrida de limpieza (02_Interim_Data_Cleaning) en MLflow UI.
Nota: Se observan el dataset vinculado, las etiquetas de gobernanza y las métricas de limpieza: 410,806 filas iniciales, 395,672 finales, 921 duraciones no positivas, 14,033 viajes menores a un minuto, 96 mayores a 120 minutos y 84 edades inválidas.

Para revisar interactivamente estas ejecuciones en el navegador, levantamos la interfaz con el siguiente comando en la terminal:

```
mlflow ui --backend-store-uri sqlite:///mlruns.db --port 5000
```

5. Estructuración Cookiecutter Data Science y Código en POO

Para evitar tener notebooks desordenados donde no se sabe qué celda correr primero, organizamos el proyecto con la plantilla estándar de Cookiecutter Data Science. Separamos la lógica en módulos independientes dentro de la carpeta src, lo que facilita el mantenimiento y las pruebas:

```
Tarea_4_Equipo/
├── data/
│   ├── raw/                  <- Datasets originales inmutables y catálogo de estaciones
│   ├── interim/              <- Datos limpios tras validación de reglas de negocio
│   └── processed/            <- Matriz con variables para modelado (267,840 filas, 16
columnas)
├── notebooks/
│   └── 01_eda_mibici.ipynb   <- Cuaderno Jupyter de análisis exploratorio inicial
├── src/
│   ├── data/                 <- Módulos POO de descarga (downloader.py) y limpieza
(cleaner.py)
│   ├── features/             <- Ingeniería de características (engineer.py) y
preprocesamiento (preprocessor.py)
│   ├── models/               <- Baselines registrados en MLflow (train_baseline.py)
│   └── utils/                <- Logger de gobernanza y versionado con MLflow
(mlflow_logger.py)
├── tests/
│   └── test_pipeline.py      <- Suite de pruebas automatizadas con unittest
├── reports/
│   ├── figures/              <- Gráficas del EDA y capturas de MLflow UI
│   ├── ML_Canvas_MiBici.md   <- Documentación del Machine Learning Canvas
│   ├── roles_mlops.md        <- Matriz de responsabilidades del equipo
│   └── Fase 1_Equipo19.pdf   <- Reporte ejecutivo oficial de entrega
├── requirements.txt          <- Lista de dependencias del entorno Python
├── environment.yml           <- Archivo de configuración reproducible para Conda
└── README.md                 <- Explicación general y guía rápida para correr el código
```

Pruebas Unitarias Automatizadas (tests/test_pipeline.py)

Para validar que el código de limpieza y transformación funcione siempre bien, creamos una suite de pruebas con la librería unittest que se ejecuta sobre un dataset pequeño y controlado en un directorio temporal, sin modificar los datos del proyecto:

• test_cleaner_removes_injected_anomalies: Sobre datos de prueba con anomalías inyectadas (duplicados, duraciones negativas, viajes cortos y largos, estaciones y edades inválidas), comprueba que se descarte exactamente cada caso. (ok)

• test_cleaner_duration_bounds: Verifica que los viajes se encuentren dentro del rango de 1 a 120 minutos. (ok)

• test_cleaner_valid_stations: Revisa que todas las estaciones de inicio y fin existan en el catálogo. (ok)

• test_feature_engineer_columns: Confirma que la matriz final tenga exactamente las 16 columnas esperadas. (ok)

• test_cyclical_bounds: Verifica que los valores de seno y coseno no se salgan del rango de -1 a 1. (ok)

• test_demand_grid_preserves_trips: Valida que la cuadrícula estación-hora conserve el total de viajes y no tenga demandas negativas. (ok)

• test_temporal_split_is_chronological: Verifica que entrenamiento, validación y prueba no se traslapen en el tiempo. (ok)

• test_scaler_fit_only_on_train: Verifica que el escalador se ajuste únicamente con datos de entrenamiento. (ok)

• test_pca_reaches_target_variance: Verifica que PCA conserve al menos el 95% de la varianza. (ok)

Resultado de las pruebas: Ran 9 tests in 1.166s - OK (todas aprobadas exitosamente).

6. Definición de Roles MLOps e Interacción del Equipo

Siguiendo los lineamientos de la rúbrica, nos organizamos asignando a cada compañero un rol específico dentro de las actividades de esta primera fase:

Rol MLOps	Integrante Asignado	Área de Enfoque	Aporte en la Fase 1
Data Engineer (DE)	José Antonio Vázquez Martínez (A01797208)	Ingesta y Calidad de Datos	Programó la descarga y limpieza de datos en downloader.py y cleaner.py, eliminó viajes inconsistentes y registró las versiones raw e interim en MLflow.
Data Scientist (DS)	Jesús Ángel Rincón Ruiz (A01793960)	EDA y Feature Engineering	Completó la propuesta en el ML Canvas, analizó los picos horarios en el notebook de EDA y diseñó las variables cíclicas y de rezago temporal.
Software Engineer (SWE)	Erick Francisco Siller Ojeda (A01382929)	Arquitectura y Calidad de Código	Estructuró las carpetas bajo Cookiecutter Data Science, refactorizó los scripts a clases de orientación a objetos y armó la suite de pruebas unitarias.
Site Reliability Engineer (SRE)	Gustavo Alejandro Vázquez Rivas (A01611559)	Entornos y Persistencia	Armó los archivos requirements.txt y environment.yml, configuró el almacenamiento local en SQLite para MLflow y verificó la estabilidad del entorno.
Machine Learning Engineer (MLE)	José Antonio Vázquez Martínez (A01797208)	Gobernanza y Orquestación	Conectó el logger de MLflow con hashes SHA-256, revisó el flujo completo de punta a punta y dejó las entradas listas

para los modelos de la Fase 2.

Interacción del equipo: Trabajamos de manera conectada en cada paso. Primero, el SRE y SWE dejaron listo el entorno de trabajo y la estructura de carpetas en Git. Luego, con esa base, el Data Scientist y el Data Engineer establecieron los criterios de limpieza y las metas en el ML Canvas. En cuanto los datos quedaron limpios y versionados en MLflow, el Data Scientist realizó la exploración gráfica y la ingeniería de características. Finalmente, el Software Engineer y el ML Engineer integraron los tests automatizados para validar que todo el pipeline funcionara de inicio a fin sin errores.

7. Repositorio del Proyecto y Referencias

Repositorio de GitHub: https://github.com/AntonioVazquezMartinez/Operacion-de-aprendizaje-automatico---Proyecto-Bici

En el repositorio se puede consultar el historial de commits colaborativos, la estructura completa del proyecto, el código modular en Python, el cuaderno interactivo de EDA y los archivos de configuración.

Reflexión

Al terminar esta primera fase nos dimos cuenta del cambio tan grande que representa trabajar con buenas prácticas de MLOps en comparación con un proyecto común. Normalmente uno suele hacer todo en un solo cuaderno de Jupyter, probando celdas hacia arriba y hacia abajo, pero cuando se trabaja en equipo eso se vuelve un descontrol muy rápido. Tener la estructura de Cookiecutter Data Science nos ayudó a poner cada cosa en su lugar desde el principio, separando los datos originales de los limpios y dejando los scripts modulares listos para reutilizarse.

La parte de MLflow nos pareció de las más útiles de toda la práctica. Saber que cada transformación de datos queda guardada con su fecha, su autor, sus métricas y sobre todo su hash SHA-256 da una certeza tremenda. Ya no existe la duda de saber con qué archivo exacto se generaron las variables para el modelo. Además, dividir el trabajo entre los diferentes roles nos permitió entender que llevar Machine Learning a producción no es únicamente tarea de quien diseña el algoritmo, sino que requiere ingeniería de software, infraestructura confiable y seguimiento riguroso en cada etapa.

Referencias Bibliográficas

• MiBici (2024). Datos Abiertos del Sistema de Bicicletas Públicas del Área Metropolitana de Guadalajara. Instituto de Información Estadística y Geográfica de Jalisco (IIEG). https://www.mibici.net/es/datos-abiertos/

• Dorard, L. (2020). Machine Learning Canvas: A framework to define and communicate ML systems. https://www.ownml.co/machine-learning-canvas

• DrivenData (2023). Cookiecutter Data Science: A logical, reasonably standardized, but flexible project structure for doing and sharing data science work. https://drivendata.github.io/cookiecutter-data-science/

• Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. IEEE Micro, 38(5), 39-45.

• Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J. F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS 2015), 28, 2503-2511.